

E-CONTENT: CARTESIAN PRODUCT IN SQL

Class XII CBSE Computer Science

Interactive • Conceptual • Simulation-based • CBSE-oriented

Teacher/School: _____ Teacher: _____

Suggested duration: 40–50 minutes

1. Learning Objectives

- Define Cartesian Product in SQL.
- Explain how two tables are combined to generate all possible row combinations.
- Calculate the degree and cardinality of a Cartesian Product.
- Write Cartesian Product queries using comma syntax and CROSS JOIN.
- Distinguish Cartesian Product from conditional joins.
- Predict SQL output and solve CBSE-style questions.

2. Engage: Classroom Analogy

Imagine 3 students and 2 subjects. If every student is paired with every subject, the result contains every possible student–subject combination. This is the basic idea of a Cartesian Product.

Student	Subject
Anu	Computer Science
Anu	Mathematics
Ravi	Computer Science
Ravi	Mathematics
Meera	Computer Science
Meera	Mathematics

3. What is Cartesian Product?

The Cartesian Product of two relations produces a new relation containing all possible combinations of rows from the two relations.

SQL syntax:

```
SELECT *  
FROM Table1, Table2;
```

Modern explicit syntax:

```
SELECT *  
FROM Table1  
CROSS JOIN Table2;
```

4. The Most Important Formula

If Table A has m rows and Table B has n rows:

Cardinality of Cartesian Product = $m \times n$

If Table A has p columns and Table B has q columns:

Degree of Cartesian Product = $p + q$

Example: A has 4 rows and 3 columns; B has 5 rows and 2 columns. Result: Cardinality = $4 \times 5 = 20$ rows; Degree = $3 + 2 = 5$ columns.

5. Interactive Simulation

STUDENT

RollNo	Name
1	Anu
2	Ravi
3	Meera

SUBJECT

SubCode	Subject
C01	Computer
M01	Mathematics

Prediction: How many rows will the Cartesian Product contain? _____

Answer: $3 \times 2 = 6$ rows.

6. First SQL Query

```
SELECT *  
FROM STUDENT, SUBJECT;
```

This produces all possible combinations. There is no matching condition between the two tables.

```
SELECT *  
FROM STUDENT  
CROSS JOIN SUBJECT;
```

7. Cartesian Product vs JOIN

Cartesian Product

Produces all possible row combinations

No matching condition is required

Can produce a very large result

FROM A, B / CROSS JOIN

Also called CROSS JOIN

JOIN

Combines rows based on a condition or defined relationship

Usually uses a matching condition

Usually produces fewer relevant rows

JOIN ... ON ...

Includes equi-join, natural join etc.

8. Cartesian Product with WHERE

A traditional SQL join can be understood as a Cartesian Product followed by filtering with a WHERE condition.

```
SELECT *  
FROM STUDENT, SUBJECT  
WHERE STUDENT.SubjectCode = SUBJECT.SubCode;
```

Conceptual flow:

Cartesian Product → WHERE condition → Matching rows

9. Quick Interactive Prediction

Table A has 3 rows. Table B has 2 rows.

Query:

```
SELECT *  
FROM A, B;
```

How many rows? A. 3 B. 5 C. 6 D. 9

Answer: C. 6

10. Degree vs Cardinality

Degree = number of columns/attributes. Cardinality = number of rows/tuples.

Example: A has 4 columns and 10 rows. B has 3 columns and 5 rows.

Degree = $4 + 3 = 7$ columns.

Cardinality = $10 \times 5 = 50$ rows.

11. Quick Assessment

Q1. Table A has 5 rows and Table B has 4 rows. How many rows will their Cartesian Product contain? _____

Q2. Table A has 3 columns and Table B has 5 columns. What will be the degree? _____

Q3. If A has 10 rows and B has 20 rows, how many records can `SELECT * FROM A, B;` produce? _____

Answers: Q1 = 20; Q2 = 8; Q3 = 200.

12. Sample Query: Selected Columns

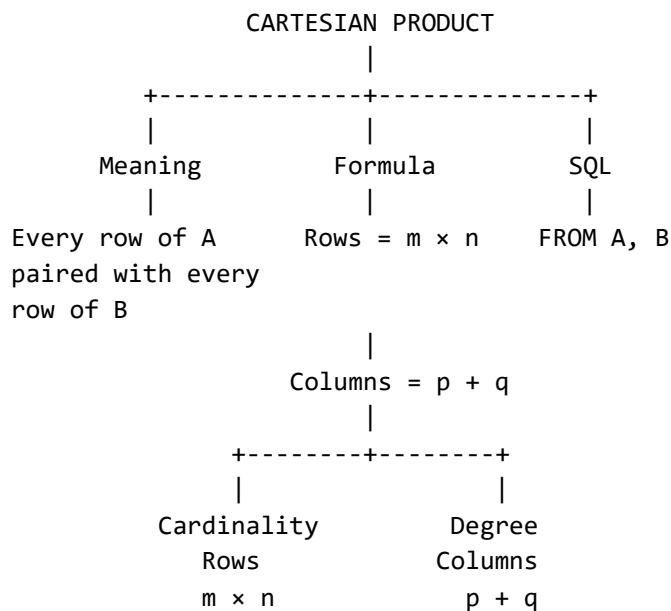
```
SELECT STUDENT.Name, SUBJECT.Subject
FROM STUDENT, SUBJECT;
```

Selecting fewer columns changes what is displayed, but it does not change the underlying number of row combinations.

13. Why Can Cartesian Product Be Dangerous?

If Table A contains 1,000 rows and Table B contains 2,000 rows, their Cartesian Product can contain $1,000 \times 2,000 = 2,000,000$ combinations. An accidental missing join condition can therefore create a very large result.

14. Mind Map



15. CBSE Exam Pattern

CBSE curriculum material includes Cartesian Product with joins, and CBSE Computer Science resources include questions involving degree/cardinality and output of Cartesian Product. For current board preparation, teachers should verify exact wording and marking requirements against the latest official CBSE Sample Question Paper, Marking Scheme and previous-year papers.

16. CBSE-Style Question 1

A table STUDENT1 has 4 columns and 10 rows. Another table STUDENT2 has 5 columns and 7 rows. If their Cartesian Product is performed, determine the degree and cardinality.

Answer: Degree = $4 + 5 = 9$; Cardinality = $10 \times 7 = 70$.

17. CBSE-Style Question 2

CUSTOMER has 2 rows and 2 columns. PRODUCT has 3 rows and 2 columns. Write the Cartesian Product query and determine the number of rows and columns.

```
SELECT *  
FROM CUSTOMER, PRODUCT;
```

Answer: 6 rows and 4 columns.

18. CBSE-Style Question 3: Output Prediction

CUSTOMER: Anu, Ravi. PRODUCT: Laptop, Printer.

```
SELECT CUSTOMER.Name, PRODUCT.Product  
FROM CUSTOMER, PRODUCT;
```

Expected output:

Name	Product
Anu	Laptop
Anu	Printer
Ravi	Laptop
Ravi	Printer

19. Challenge Question

EMPLOYEE has 5 columns and 12 rows. DEPARTMENT has 3 columns and 4 rows. Query: `SELECT * FROM EMPLOYEE, DEPARTMENT;`

Questions: 1) Degree? 2) Cardinality? 3) How many combinations?

Answer: Degree = 8; Cardinality = 48; combinations = 48.

20. Common Student Mistakes

- Cartesian Product multiplies the number of rows; it does not add them.
- Degree is added, not multiplied.
- Degree means columns; cardinality means rows.
- Every row of the first table is paired with every row of the second table.
- An unnecessary Cartesian Product can create a very large result.

21. Exit Ticket

1. What does Cartesian Product do? _____
2. Write its SQL syntax. _____
3. If A has 6 rows and B has 8 rows, how many rows result? _____
4. If A has 4 columns and B has 3 columns, what is the degree? _____
5. What is one difference between Cartesian Product and a conditional JOIN? _____

22. One-Minute Revision

CARTESIAN PRODUCT

Every row of A × Every row of B

Rows = rows(A) × rows(B)

Columns = columns(A) + columns(B)

SQL:

```
SELECT * FROM A, B;
```

Modern equivalent:

```
SELECT * FROM A CROSS JOIN B;
```